Software Test Plan (STP) - Hostel Management System

Project: Hostel Management System (HMS)

Version: 2.0 (Final)

Authors: Group G-03 - Tejasvi K S, Prathiksha B P, Vilohith, Varsha

Date: 08-11-2026

Status: Final, all test cases executed at the end of Sprint 2

Revision History

Version	Date	Author	Change Summary
1.0	12-10-2026	Team	Test plan and 18 test cases submitted for Jackfruit Part 1
2.0	08-11-2026	Team	Final version after Sprint 2: test environment and tools updated to match the project (GitHub Actions, SonarQube Cloud, Locust), 11 test cases added for the requirements marked Planned, actual results recorded for all test cases, test summary report added (section 17)

1. Introduction

- **Purpose:** This document defines the test plan for the Hostel Management System (HMS) v2.0: objectives, scope, strategy, environment, schedule and responsibilities, and records the results of executing it.
- **Scope:** Testing covers registration with admin approval, login, role-based access, profile, room allocation and deallocation, fee and payment recording, fee details and dues report, complaints, leave requests with admin approval, session security, input security, audit logging and the non-functional requirements. Exclusions are listed in section 4.
- **References:** HMS SRS v2.0, HMS SAD v2.0, IEEE 829 (test documentation), OWASP ASVS.
- **Definitions:** HMS (Hostel Management System), SRS (Software Requirements Specification), SAD (Software Architecture and Design), RTM (Requirements Traceability Matrix), RBAC (Role-Based Access Control), CI (Continuous Integration).

2. Test Items

- Accounts & Access module (registration, approval, login, roles, profile)
- Room Allocation module (blocks, rooms, allocation, deallocation, occupancy)
- Fee & Payment module (fee amount, payment records, outstanding balance, fee details, dues report)
- Complaints & Leave module (complaints, complaint status, leave requests, leave decisions)
- Audit module (audit log) and Security module (session settings)
- Role-based navigation and admin interface

3. Features to be Tested

Features are mapped to SRS requirement IDs.

3.1 Accounts & Access

- REQ-1: Student registration with unique email and SRN

- REQ-2: New accounts stay Pending until verified
- REQ-3: Admin approves or rejects a registration
- REQ-4: Role-based access to student and admin pages
- REQ-5: Student views and edits own profile
- REQ-21: Login with email or SRN and password
- SEC-1 to SEC-6: Password hashing, role-based access, input validation, session expiry, CSRF protection, audit log

3.2 Room Allocation

- REQ-6: Define blocks and rooms with a capacity
- REQ-7: Prevent allocation beyond capacity
- REQ-8: Allocate a room to a Verified student
- REQ-9: Deallocate a student from a room
- REQ-10: Automatic occupancy and status update
- NFR-1: Response time of standard pages under load

3.3 Fee & Payment

- REQ-11: Admin defines a fee amount for a student
- REQ-12: Admin records a payment
- REQ-13: Outstanding balance is fee minus total payments
- REQ-14: A payment cannot exceed the outstanding balance
- REQ-15: Student views own fee, payments and outstanding balance
- REQ-16: Admin views the outstanding dues report

3.4 Complaints & Leave

- REQ-17: Student files a complaint for their allocated room
- REQ-18: Admin updates complaint status
- REQ-19: Student submits a leave request with a valid date range
- REQ-20: Admin approves or rejects a pending leave request

3.5 Non-functional

- NFR-1 performance: response time of standard pages
- NFR-2 usability: number of screens to file a complaint
- NFR-3 reliability: failed or invalid submissions change nothing; multi-step updates are atomic
- NFR-4 maintainability: modular apps and at least 70% line coverage per module
- NFR-5 compatibility: latest Chrome and Firefox
- Security: see section 5.1

4. Features Not to be Tested

- Django framework and PostgreSQL internals (assumed tested by their projects)
- Hosting, server and network infrastructure, HTTPS configuration
- Online payment gateways, email and SMS delivery (out of scope in the SRS)
- Browsers other than the latest Chrome and Firefox
- Third-party libraries beyond their use in HMS
- Penetration testing with OWASP ZAP (planned in v1.0, not performed; security is covered by the automated tests in section 5.1)

5. Test Approach / Strategy

Levels - Unit tests: service functions with pytest (for example outstanding balance, capacity check, decision rules) - Integration tests: views, forms, services and database together with the Django test client - System tests: complete flows through the pages (for example register, approve, allocate, file complaint), automated with the Django test client and walked through manually in the browser - Acceptance tests: demo walkthrough of all use cases by the team and faculty

Types - Functional testing of every REQ - Negative and boundary testing (invalid input, full room, payment above outstanding, exact outstanding amount) - Regression testing: the full automated suite (168 tests) runs on PostgreSQL in GitHub Actions on every pull request and every push to `main` - Performance testing against NFR-1 with Locust - Usability testing against NFR-2 (screen count, automated and manual) - Security testing (see section 5.1) - Static analysis with SonarQube Cloud on every pull request

Test data: each automated test creates its own data (students, admin, blocks, rooms, fees) in its set-up method, so tests do not depend on each other. The load test uses `loadtest/seed.py` to create 50 rooms in 5 blocks, 1 admin and 10 verified students with fees.

Entry criteria: build passes CI, the Docker environment runs, SRS v1.3 is the baseline (final version v2.0).

Exit criteria: all planned test cases are executed; no open critical or high defects; at least 70% line coverage per module (NFR-4); SonarQube Cloud quality gate passed with no open vulnerabilities; every requirement in the RTM has at least one executed test case. **All exit criteria were met on 08-11-2026** (section 17).

5.1 Security Validation

Area	Requirement	How it is validated	Test case	Result
Password handling	SEC-1	The stored password is a hash, not the plain text; failed logins are logged without the password	TC-SEC-06, TC-SEC-04	Pass
Access control	SEC-2, REQ-4	Every admin page opened as a Student and every student page opened as an Admin gives 403; a Pending student gets 403; visitors are sent to login; a student cannot open another student's records by changing the URL	TC-SEC-01, TC-FEE-03	Pass
Input validation	SEC-3	Script tags in complaints, names and leave reasons are shown escaped; SQL text is stored as plain text or rejected; over-long values, bad phone numbers and text in amount fields are rejected	TC-SEC-03	Pass
Session management	SEC-4	Expired, logged-out and tampered sessions are sent to login; idle timeout is 30 minutes; each request extends it; cookie is HttpOnly; new session key after login	TC-SEC-02	Pass
CSRF	SEC-5	A POST without a CSRF token gets 403 and nothing is saved	TC-SEC-05	Pass
Audit log	SEC-6	Failed login, allocation, deallocation, fee, payment, decisions and 403s create audit entries with no password or token	TC-SEC-04	Pass
Static analysis	All	SonarQube Cloud scan on every pull request	-	Quality gate passed, 0 vulnerabilities, 0 security hotspots

The penetration test with OWASP ZAP listed in v1.0 was not performed; the automated security tests above cover the same login, registration and form flows.

6. Test Environment

- **CI (GitHub Actions):** Ubuntu runner, Python 3.12, PostgreSQL 16 service container, dependencies from `requirements.lock`; runs Django system checks, the migration check, the full test suite with coverage, and the per-module coverage check.
- **Developer laptops:** Windows, Python 3.12, local SQLite database or Docker Compose (application and PostgreSQL 16).
- **Browsers:** Chrome 156.0.8078.80 and Firefox 159.0.1 (NFR-5).
- **Tools:** pytest, pytest-django and the Django test client (unit, integration, system), coverage.py (NFR-4), Locust (performance), SonarQube Cloud (static analysis), GitHub Actions (CI), GitHub Issues and GitHub Projects (backlog and defect tracking).
- **Test data:** created by each test (section 5); load-test data from `loadtest/seed.py`.

7. Test Schedule

Milestone	Date
Test case design	09-Oct-2026 to 10-Oct-2026
Environment setup (Docker, CI)	Sprint 1
Test plan review and Part 1 submission	11-Oct-2026 to 12-Oct-2026
Sprint 1: unit and integration tests written with each feature (pull requests #78 to #81)	25-Oct-2026 to 03-Nov-2026
Sprint 2: remaining feature, security, reliability, usability and coverage tests (pull requests #82 to #89)	06-Nov-2026 to 08-Nov-2026
Performance test (Locust), browser checklist, usability walkthrough	06-Nov-2026 to 08-Nov-2026
Final test run and test summary report	08-Nov-2026

8. Test Deliverables

Deliverable	Location
Test Plan	`docs/STP.md` (this document)
Test cases	Section 16
Automated test scripts	`<module>/tests/` and `tests/` (168 tests)
Load test script and seed data	`loadtest/locustfile.py`, `loadtest/seed.py`
Performance results (NFR-1)	`docs/testing/performance_results.md`
Usability walkthrough (NFR-2)	`docs/testing/usability_nfr2.md`
Browser compatibility checklist (NFR-5)	`docs/testing/browser_compatibility.md`
Test execution logs and coverage report	GitHub Actions runs (CI job log and `coverage-report` artifact)
Static analysis report	SonarQube Cloud project dashboard
Defect reports	GitHub Issues
Test Summary Report	Section 17

9. Roles and Responsibilities

Role	Name	Responsibility
Test Lead	Vilohith	Metrics, CI test runs, defect tracking, test summary report
Security Test Owner	Tejasvi K S	Security validation (section 5.1), SonarQube Cloud, input security, reliability and coverage tests
Integration and Performance Test Owner	Prathiksha B P	Test environment, room allocation tests, audit tests, performance and browser tests
STP Editor and Requirements Traceability	Varsha	Traceability matrix, complaints and leave tests, session tests, usability test, final document
Product Owner / Approver	Faculty	Reviews and approves the plan and results

10. Risks and Mitigation

Risk	Mitigation
Delay in a stable build	Small pull requests merged often, full test suite on every CI run
Test environment differs between laptops	Single Docker setup shared by the whole team; CI always tests on PostgreSQL
Not enough time to execute all cases	High priority requirements first, unit and regression tests automated
Inconsistent test data	Each test creates its own data; load-test data from one seed script

11. Assumptions & Dependencies

- SRS v1.3 is the baseline; the final requirements are in SRS v2.0
- Docker is available on team members' laptops, or the local SQLite setup is used
- GitHub Actions and SonarQube Cloud are reachable during the implementation phase

12. Suspension & Resumption Criteria

Suspend testing if: - The test environment is unavailable for more than 4 hours - The build fails CI, or blocks more than 30% of the planned test cases - A critical defect in login blocks all other tests

Resume testing when: - Blocking defects are fixed and CI passes - The environment is restored

13. Test Case Management & Traceability

The Requirements Traceability Matrix (RTM) maps SRS requirements to test cases. Test cases are listed in section 16. Every requirement has at least one executed test case.

Requirement	Test cases	Status
REQ-1	TC-ACC-03	Pass
REQ-2	TC-ACC-04	Pass
REQ-3	TC-ACC-04	Pass
REQ-4	TC-SEC-01	Pass
REQ-5	TC-ACC-05	Pass
REQ-6	TC-ROOM-05	Pass

Requirement	Test cases	Status
REQ-7	TC-ROOM-02	Pass
REQ-8	TC-ROOM-01, TC-ROOM-03	Pass
REQ-9	TC-ROOM-04	Pass
REQ-10	TC-ROOM-01, TC-ROOM-04	Pass
REQ-11	TC-FEE-01	Pass
REQ-12	TC-FEE-01	Pass
REQ-13	TC-FEE-01	Pass
REQ-14	TC-FEE-02	Pass
REQ-15	TC-FEE-03	Pass
REQ-16	TC-FEE-04	Pass
REQ-17	TC-REQ-01	Pass
REQ-18	TC-REQ-02	Pass
REQ-19	TC-REQ-03	Pass
REQ-20	TC-REQ-04	Pass
REQ-21	TC-ACC-01, TC-ACC-02, TC-ACC-04	Pass
SEC-1	TC-SEC-06	Pass
SEC-2	TC-SEC-01	Pass
SEC-3	TC-SEC-03	Pass
SEC-4	TC-SEC-02	Pass
SEC-5	TC-SEC-05	Pass
SEC-6	TC-SEC-04	Pass
NFR-1	TC-PERF-01	Pass
NFR-2	TC-USE-01	Pass
NFR-3	TC-REL-01	Pass
NFR-4	TC-COV-01	Pass
NFR-5	TC-BRW-01	Pass

14. Test Metrics & Reporting

Metrics collected - % of test cases executed - % passed and failed - Requirement coverage (requirements with at least one executed test case) - Defects found and open - Line coverage per module

Reports - CI result on every pull request (tests, coverage per module, SonarQube Cloud quality gate) - Final Test Summary Report (section 17)

15. Approvals

Role	Name	Signature / Date
Test Lead	Vilohith	
Dev Lead	Tejasvi K S	

Role	Name	Signature / Date
Faculty		

16. Test Cases

Each test case lists the requirement it verifies, its level (Unit, Integration or System), pre-conditions, steps, test data and the expected result. Test cases TC-ACC-01 to TC-USE-01 are from v1.0; TC-ACC-05 onwards were added in Sprint 2 for the requirements marked Planned in v1.0. Actual results are from the final run on 08-11-2026; the automated test that executes each case is named in the Actual Result column.

16.1 Accounts & Access

Test Case ID	Requirement	Level	Description	Pre-conditions	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC-ACC-01	REQ-21	System	Verified student logs in	Verified student account exists	1) Open login page 2) Enter email and password 3) Submit	student1@example.com / Pass@1234	Redirected to the student dashboard	Redirected to the student dashboard; login with SRN also works (`test_verified_student_logs_in_with_email` , `test_verified_student_logs_in_with_srn`)	Pass
TC-ACC-02	REQ-21	System (negative)	Wrong password is rejected	Verified student account exists	1) Open login page 2) Enter valid email 3) Enter wrong password 4) Submit	student1@example.com / wrongpass	Login refused with an Invalid credentials message; no session is created	Login refused with an error message, no session created (`test_wrong_password_is_rejected`)	Pass
TC-ACC-03	REQ-1	System (negative)	Duplicate email or SRN is rejected	student1@example.com and SRN PES1UG24CS001 already registered	1) Register with the same email and a new SRN 2) Register with a new email and the same SRN	student1@example.com / new SRN, then new email / PES1UG24CS001	Both registration attempts are rejected with an already registered message; no duplicate user is created	Both refused with an "already registered" error, in any letter case; no user created (`test_duplicate_email_is_rejected` , `test_duplicate_srn_is_rejected`)	Pass
TC-ACC-04	REQ-2, REQ-3, REQ-21	Integration	New account cannot log in until the Admin approves it	Admin account exists	1) Register a new student 2) Try to log in 3) Admin approves the registration 4) Log in again	student9@example.com	Step 2 is refused because the account is Pending; step 4 succeeds and the account status is Verified	Pending and Rejected accounts refused; after approval the status is Verified and login succeeds (`test_pending_account_cannot_log_in` , `test_admin_approves_registration`)	Pass

Test Case ID	Requirement	Level	Description	Pre-conditions	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC-ACC-05	REQ-5	Integration	Student views and edits own profile	Verified student logged in	1) Open My Profile 2) Change the name and save 3) Clear the name and save 4) Enter "abc" as guardian contact and save	New name, empty name, "abc"	Step 2 saved with a success message; steps 3 and 4 refused with a field error; SRN and email cannot be changed; no way to open another student's profile	As expected (`test_student_updates_name_and_guardian_contact` , `test_invalid_update_is_rejected` , `test_srn_and_email_cannot_be_changed` , `test_student_cannot_edit_another_profile`)	Pass
TC-SEC-01	REQ-4, SEC-2	System (security)	Student cannot open an admin page	Logged in as Student	1) Request the admin rooms URL directly	Student session, actual admin URL	Access is denied with a 403 response or redirected to login; no admin data is exposed	403 on every admin page for a Student, 403 on every student page for an Admin, 403 for a Pending student, login redirect for visitors (`test_authorization.py` , 6 tests)	Pass

16.2 Room Allocation

Test Case ID	Requirement	Level	Description	Pre-conditions	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC-ROOM-01	REQ-8, REQ-10	Integration	Allocate an available room to a Verified student	Verified student without a room; room with capacity 2 and 0 occupied	1) Log in as Admin 2) Open allocation 3) Select student and room 4) Submit	Student S1, Room B1-101	Allocation saved, occupancy becomes 1, S1 shows room B1-101	Allocation saved, occupancy 1, student sees the room; room becomes Occupied when full (`test_allocate_available_room` , `test_room_becomes_occupied_when_full`)	Pass
TC-ROOM-02	REQ-7	Unit (boundary)	Allocation to a full room is rejected	Room with capacity 2 and 2 occupied	1) Log in as Admin 2) Try to allocate another Verified student to the room	Student S3, Room B1-102	Rejected with room is full message, occupancy stays 2	Rejected with "room is full", occupancy unchanged (`test_full_room_is_rejected` , `test_allocation_page_shows_error_for_full_room`)	Pass
TC-ROOM-03	REQ-8	Unit (negative)	A Pending student cannot be allocated a room	Student S4 with status Pending; room available	1) Log in as Admin 2) Try to allocate S4	Student S4, Room B1-103	Rejected, no allocation created	Rejected, no allocation created (`test_pending_student_is_rejected`)	Pass

Test Case ID	Requirement	Level	Description	Pre-conditions	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC-ROOM-04	REQ-9, REQ-10	Integration	Deallocate a student from a room	S1 allocated to B1-101 (occupied 1)	1) Log in as Admin 2) Deallocate S1	Student S1	Allocation removed, occupancy becomes 0, room shows Vacant, S1 dashboard shows No room allocated yet	As expected (`test_deallocate_reduces_occupancy` , `test_full_room_becomes_vacant_after_deallocation` , `test_student_sees_no_room_after_deallocation`)	Pass
TC-ROOM-05	REQ-6	Integration	Admin defines blocks and rooms with capacity	Admin logged in	1) Add block A 2) Add room 101, Double, capacity 2 3) Add a room with capacity 0 4) Add room 101 again in block A	Block A, room 101	Steps 1 and 2 saved and listed as A-101, 0 / 2, Vacant; steps 3 and 4 refused with a field error	As expected (`test_admin_creates_block_and_room` , `test_invalid_capacity_is_rejected` , `test_duplicate_room_number_in_block_is_rejected`)	Pass
TC-PERF-01	NFR-1	Performance	Standard page response time under load	50 rooms seeded	1) Run Locust with 10 concurrent users for 1 minute against the dashboard, room list and fee pages	10 users, 1 minute	95% of responses within 2 seconds	391 requests, 0 failures; 95th percentile 52 ms for all requests, 22 ms for the room list (`docs/testing/performance_results.md`)	Pass

16.3 Fee & Payment

Test Case ID	Requirement	Level	Description	Pre-conditions	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC-FEE-01	REQ-11, REQ-12, REQ-13	Unit	Recording a payment reduces the outstanding balance	Student S1 with fee 50000 defined and nothing paid	1) Log in as Admin 2) Record a payment 3) Open S1's dues	S1, amount 20000	Payment saved, outstanding shows 30000	Payment saved, outstanding 30000, also through the Fees page (`test_payment_reduces_outstanding` , `test_admin_sets_fee_and_records_payment`)	Pass
TC-FEE-02	REQ-14	Unit (boundary)	A payment above the outstanding balance is rejected, the exact amount is accepted	S1 outstanding 30000	1) Record 30001 2) Record 30000	30001 then 30000	First refused with a message and no record created; second accepted and outstanding becomes 0	As expected; the page shows a red banner and an error under Amount (`test_payment_above_outstanding_is_rejected_exact_amount_accepted` , `test_overpayment_shows_banner_and_field_error`)	Pass
TC-FEE-03	REQ-15	System	Student sees only their own fee details	S1 and S2 have fees and payments; S1 logged in	1) Open Fee Details 2) Change the URL to open S2's dues	S1, S2	Own fee, payments and outstanding shown; S2's details refused with 403	Own fee, payments and outstanding shown; S2's details refused with 403 (`test_student_sees_own_fee_payments_and_outstanding` , `test_other_students_detail_page_is_refused`)	Pass

Test Case ID	Requirement	Level	Description	Pre-conditions	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC-FEE-04	REQ-16	Integration	Admin views the dues report	Several students with fees, one fully paid	1) Log in as Admin 2) Open Dues Report 3) Tick "Show only students with dues"	Students with different balances	Every student with a fee listed, largest balance first, correct totals; the filter hides fully paid students; students cannot open the report	As expected (`test_report_totals` , `test_largest_balance_first` , `test_hide_paid_filter` , `test_student_cannot_open_report`)	Pass
TC-SEC-02	SEC-4	System (security)	Expired session is rejected	S1 logged in, session then expired	1) Expire the session (clear the server session or set a past expiry) 2) Open S1's dues page	S1 session	Redirected to login or 401, no dues data shown	Redirected to login, no dues data shown; logged-out and tampered sessions also refused (`test_sessions.py` , 9 tests)	Pass

16.4 Complaints & Leave

Test Case ID	Requirement	Level	Description	Pre-conditions	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC-REQ-01	REQ-17	System	Student with an allocated room files a complaint	S1 logged in and allocated a room	1) Open complaints 2) Enter description 3) Submit	Tap leaking	Complaint saved with status Open, linked to S1's room and shown under My complaints	As expected; a student without a room cannot file a complaint (`test_student_with_room_files_complaint` , `test_student_without_room_cannot_file_complaint`)	Pass
TC-REQ-02	REQ-18	Integration	Admin updates complaint status	Open complaint from S1	1) Log in as Admin 2) Set status In-progress 3) Set status Resolved 4) Log in as S1 and open complaints	Complaint C1	Status changes at each step and S1 sees Resolved	As expected (`test_admin_moves_complaint_through_statuses`)	Pass
TC-REQ-03	REQ-19	Unit (negative)	Leave with end date before start date is rejected	Student S1 logged in	1) Open leave form 2) Enter dates 3) Submit	from 20-Oct-2026, to 18-Oct-2026	Rejected with validation message, no record created	Rejected with a validation message, no record created (`test_end_before_start_is_rejected`)	Pass
TC-REQ-04	REQ-20	Integration	Admin approves a pending leave and cannot decide it twice	Pending leave request from S1	1) Log in as Admin 2) Approve the request 3) Try to reject the same request	Leave L1	First step: status Approved and visible to S1; second step refused as already decided, status stays Approved	As expected (`test_approve_then_reject_is_refused`)	Pass
TC-USE-01	NFR-2	Usability	New student files a complaint in few screens	New student logged in and allocated a room	1) Observer counts screens from the dashboard to the confirmation	New student account	Done in no more than 4 screens	3 screens (4 through My Room), automated and in a manual walkthrough (`test_complaint_in_at_most_four_screens` , `docs/testing/usability_nfr2.md`)	Pass

16.5 Security, Reliability, Maintainability and Compatibility

Test Case ID	Requirement	Level	Description	Pre-conditions	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC-SEC-03	SEC-3	System (security)	Malicious and invalid input is handled safely	Student and Admin accounts	1) Submit script tags in a complaint, a name and a leave reason, then view them as Admin 2) Submit SQL text in login, complaint and SRN 3) Submit over-long text, a bad phone number and text in an amount	`<script>alert(1)</script>`, `' OR '1'='1'; DROP TABLE`	Script shown as escaped text, SQL stored as text or rejected, invalid values rejected, nothing executed	As expected (`tests/test_input_security.py`, 12 tests)	Pass
TC-SEC-04	SEC-6	Integration (security)	Security-relevant events are written to the audit log	Admin and Student accounts	1) Log in with a wrong password 2) Allocate and deallocate a room 3) Set a fee and record a payment 4) Open an admin page as a Student 5) Open the Audit Log	Wrong password "wrongpass"	Each event has an entry with user and time; the failed login stores the identifier but not the password	As expected; audit entries cannot be edited in the Django admin (`audit/tests/test_audit.py`, 14 tests)	Pass
TC-SEC-05	SEC-5	Integration (security)	POST without a CSRF token is refused	Admin logged in, CSRF checks enforced	1) Send an allocation POST without a CSRF token	Valid student and room	403, no allocation created	403, nothing saved (`test_post_without_csrf_token_is_refused`)	Pass
TC-SEC-06	SEC-1	Unit (security)	Password stored as a hash	None	1) Register a student 2) Read the stored password field	Password Pass@1234	Stored value is not the plain password and verifies correctly	Stored as a salted PBKDF2 hash (`test_password_is_stored_hashed`)	Pass
TC-REL-01	NFR-3	Integration	Failed or invalid submissions leave no partial records	Rooms, fees, students	1) Make the room status update fail during an allocation 2) Make the payment save fail 3) Submit invalid allocation, payment, fee, complaint and leave requests	Simulated failure with `mock.patch`	No allocation or payment left after a failure; invalid submissions create or change nothing	As expected (`tests/test_reliability.py`, 8 tests)	Pass
TC-COV-01	NFR-4	Static / CI	Every module reaches at least 70% line coverage	CI on `main`	1) Run the tests with coverage 2) Run the per-module coverage check	Full test suite	Each module at least 70%	accounts 94%, rooms 94%, fees 97%, complaints 95%, audit 98%, security 100%; total 95% (GitHub Actions CI)	Pass
TC-BRW-01	NFR-5	System	Core workflows work in Chrome and Firefox	Running system with sample data	1) Perform the 18 checklist actions in the latest Chrome 2) Repeat in the latest Firefox	18 actions from registration to logout	All actions work with no functional difference	All 18 actions passed in Chrome 156 and Firefox 159 (`docs/testing/browser_compatibility.md`)	Pass

17. Test Summary Report

Date of final run: 08-11-2026, GitHub Actions CI on `main` after pull request #89.

Metric	Result
Automated tests	168 run, 168 passed, 0 failed
Test cases (section 16)	29 executed, 29 passed, 0 failed
Requirement coverage	32 of 32 requirements (REQ-1 to REQ-21, SEC-1 to SEC-6, NFR-1 to NFR-5) have at least one passed test case
Line coverage	95% total; accounts 94%, rooms 94%, fees 97%, complaints 95%, audit 98%, security 100%
SonarQube Cloud	Quality gate passed; 0 bugs, 0 vulnerabilities, 0 code smells, 0 security hotspots; reliability, security and maintainability rated A; 1.3% duplication
Performance (NFR-1)	95th percentile 52 ms with 10 users, 0 failures (limit 2000 ms)
Usability (NFR-2)	3 screens (limit 4)
Open defects	0 open issues on GitHub at the end of Sprint 2

Problems found during testing and how they were resolved

Problem	Found by	Resolution
SonarQube Cloud showed 0% coverage because the coverage report was not sent to the scan	SonarQube Cloud quality gate	Tests run with coverage before the scan (pull request #83)
Load-test scripts counted in coverage and failed the 80% new-code gate	SonarQube Cloud quality gate	Load-test scripts excluded from coverage (pull request #86)
Security findings in the build configuration: unlocked dependencies, Docker image running as root and copying the whole build context	SonarQube Cloud	Hash-locked `requirements.lock`, non-root Docker user, explicit copy of app folders (pull request #87)
CI workflow file invalid after the dependency change, so CI did not run	GitHub Actions	Command quoted in the workflow file (pull request #88)

Conclusion: all exit criteria in section 5 are met. The system is ready for the final demo.